

Práctica de eliminaciones, actualizaciones y validaciones

Objetivo:

Partiendo del proyecto “Práctica de Relaciones con Sequelize”, se deberán implementar validaciones para controlador. Además, será necesario crear nuevos controladores para actualizar y eliminar.

Criterios de Evaluación

1. Presentación del código

- El código debe estar limpio, ordenado y bien indentado.
- Uso obligatorio de try-catch en todos los controladores para un manejo adecuado de errores.
- Organización correcta del proyecto en carpetas:
 - **src/config** → configuración (ej. conexión a BD).
 - **src/models** → definición de modelos Sequelize.
 - **src/routes** → definición de rutas.
 - **src/controllers** → controladores.
 - **src/middlewares** → middlewares.
- Uso exclusivo de ESM modules (import / export).
- El código debe ser funcional, modularizado y sin errores de ejecución.

2. Validaciones y lógica

- Validaciones correctamente implementadas dentro de los controladores con express-validator.
- Uso correcto de Sequelize con conexión a MySQL, definición adecuada de modelos y relaciones.
- Respuestas claras y códigos HTTP apropiados en cada operación:
 - **201 Created** → al crear.
 - **200 OK** → en consultas y actualizaciones exitosas.
 - **400 Bad Request** → errores de validación.
 - **404 Not Found** → recurso inexistente.
 - **500 Internal Server Error** → errores inesperados.
- Verificación de unicidad al crear o editar recursos con campos únicos.
- Verificación de existencia previa antes de editar o eliminar un recurso.
- Mostrar mensajes de éxito o error al realizar operaciones CRUD.

Entrega mediante Git y GitHub – Uso de ramas y commits

Requisitos obligatorios para el control de versiones.

Teniendo en cuenta que en la práctica anterior las ramas **main** y **develop** contienen el mismo contenido, ahora se debe hacer lo siguiente:

- Crear una rama a partir de “**develop**” llamada “**validaciones**”.
- Realizar el desarrollo del trabajo en la rama “**validaciones**”.
- Al finalizar el trabajo:
 - Hacer un merge limpio de “**validaciones**” hacia “**develop**”, sin conflictos.
 - Luego hacer un merge limpio de “**develop**” hacia “**main**”, para que ambas ramas queden sincronizadas nuevamente
- Durante el desarrollo en la rama **validaciones**, se deben realizar al menos 4 commits con mensajes claros y descriptivos.

Consignas:

1. Eliminación en cascada y lógica

Configurar en los modelos existentes las siguientes reglas de eliminación:

- Implementar **eliminación en cascada** en al menos dos módulos relacionados.
- Implementar **eliminación lógica** en al menos un módulo.
- Refactorizar las relaciones existentes para que soporten estos modos de eliminación.

2. Refactorización de controladores

- Para todos los modelos existentes debe existir un CRUD completo.
- Refactorizar los controladores para incluir validaciones de entrada y existencia utilizando express-validator.
- Asegurarse de que todas las rutas (POST, GET, PUT, DELETE) tengan validaciones consistentes:
 - Validar IDs recibidas tanto en el body como por params (como mínimo que sean enteros y existan en la BD)
 - Validar campos obligatorios (por ejemplo: notEmpty, isLength, etc.)
 - Agregar validaciones personalizadas (custom) según cada modelo.